PostgreSQL

ANI-RETAIL
SALES ANALYTICS SYSTEM

COMPLETE END-TO-END
POSTGRESQL PROJECT REPORT

TECHNOLOGIES USED

- POSTGRESQL
- CSV DATASET
- DATA CLEANING
- DATABASE NORMALIZATION
- RELATIONAL DATABASE DESIGN
- DATA ANALYSIS

Prepared by :
Aniket Mishra

Abstract

AniRetail is an end-to-end PostgreSQL retail analytics project developed to transform raw retail transaction data into a structured and analysis-ready relational database system. The project includes complete data cleaning, normalisation, schema design, SQL analysis, and business insight generation.

The project begins with raw retail sales data containing duplicates, inconsistent formats, missing values, and redundant information. After performing data cleaning and validation, the dataset is normalised into multiple relational tables using Third Normal Form (3NF). PostgreSQL tables are then created using primary and foreign key constraints, followed by SQL-based business analysis.

The project demonstrates practical implementation of SQL, relational database management systems, and business intelligence concepts.

1. Business Problem Statement

Retail businesses generate large volumes of transactional data every day. However, raw business data often contains duplicates, missing values, inconsistent formats, and redundant information, making analysis difficult and unreliable.

AniRetail aims to solve this problem by building a structured PostgreSQL-based retail analytics system that:

- Cleans and standardises retail sales data
- Normalises the dataset into relational tables
- Stores structured data in PostgreSQL
- Enables business analysis using SQL queries
- Helps management understand sales performance, employee contribution, and product trends

The project demonstrates a complete data pipeline from raw CSV data to analytical insights.

2. Project Objectives

The main objectives of this project are:

- Perform data cleaning on raw retail sales data
- Remove duplicates and handle missing values
- Standardise inconsistent text and date formats
- Normalise the dataset into multiple relational tables
- Create PostgreSQL tables using proper constraints
- Insert cleaned data into normalised tables
- Perform business analysis using SQL queries
- Generate insights for sales, products, and employee performance

3. Dataset Description

The dataset contains retail transaction information including:

Column Name	Description
order_id	Unique order identifier
employee_id	Employee handling the sale
employee_name	Name of employee
employee_region	Region of employee
product_id	Unique product identifier
product_name	Product name
category	Product category
quantity	Quantity sold
product_price	Price per product
sales	Total sales amount
order_date	Date of order

Screenshot – Showing Raw data and cleaned data table creation and importing data from csv file

```sql
--Create Raw Table and import data from CSV file

CREATE TABLE retail_sales_raw (
    order_id TEXT,
    customer_name TEXT,
    employee_id TEXT,
    employee_name TEXT,
    department TEXT,
    employee_region TEXT,
    salary TEXT,
    product_id TEXT,
    product_name TEXT,
    category TEXT,
    product_price TEXT,
    quantity TEXT,
    sales TEXT,
    order_date TEXT
);

SELECT * FROM retail_sales_raw;

--Creating Retail Sales Clean Table and importing raw data from Retail Sales raw table
CREATE TABLE retail_sales_clean AS
SELECT * FROM retail_sales_raw;

SELECT * FROM retail_sales_clean;
```

4. Data Cleaning Process

The raw dataset contained several issues:

- Duplicate rows
- Missing sales values
- Inconsistent capitalization
- Extra spaces in text fields
- Inconsistent category naming
- Mixed date formats

Cleaning Steps Performed:

1. Removing Duplicate Rows

Duplicate records were identified and removed to ensure data uniqueness and avoid incorrect analysis results.

Functions/Clauses Used

- COUNT(*)
- GROUP BY
- HAVING
- DELETE
- ctid

2. Removing Extra Spaces

Leading and trailing spaces from text columns were removed to maintain consistency in employee names, product names, categories, and regions.

Function Used

- TRIM()

3. Standardizing Text Case

Text values were standardized into a consistent format by converting text into lowercase and then capitalizing the first letter of each word.

Functions Used

- LOWER()
- INITCAP()

4. Fixing Inconsistent Values

Different spellings and formats representing the same category or product were standardized into a single consistent value.

SQL Operations Used

- UPDATE
- WHERE
- IN

5. Handling Missing Values

Missing values in columns such as department were identified and replaced with default values like "Unknown".

Functions/Clauses Used

- IS NULL
- UPDATE

6. Removing Special Characters

Special characters and unnecessary symbols were identified for cleaning to improve data consistency.

Possible Functions

- REPLACE()

7. Standardizing Date Formats

The dataset contained multiple date formats. All dates were converted into a single standardized PostgreSQL DATE format.

Functions Used

- CASE
- TO_DATE()

- Regular Expressions (~)
- ALTER TABLE

8. Fixing Incorrect Datatypes

Initially, all columns were stored as text datatype. Appropriate datatypes such as INT, NUMERIC, and DATE were assigned to improve database performance and analysis accuracy.

Functions Used

- ALTER TABLE
- ALTER COLUMN
- USING
- Type Casting (::)

9. Checking Negative and Invalid Values

Sales values were recalculated using product price and quantity. Invalid negative sales values were identified and corrected.

Sales formula used:

Sales = Product_Price × QuantitySales

QuantitySales = Product_Price× Quantity

Final Outcome

After completing the data cleaning process:

- Duplicate records were removed
- Missing values were handled
- Text formatting was standardized
- Date formats were normalized

- Datatypes were corrected
- Invalid values were fixed

The final cleaned dataset became structured, consistent, and analysis-ready for database normalization and SQL-based business analysis.

Screenshots:

```sql
--TYPE 1: DUPLICATE ROWS

--FIND DUPLICATES
SELECT order_id ,
   COUNT(*) FROM retail_sales_clean GROUP BY order_id
   HAVING COUNT(*)>1;

--REMOVING DUPLICATES
DELETE FROM retail_sales_clean first_table
USING retail_sales_clean second_table
WHERE first_table.ctid < second_table.ctid
AND first_table.order_id = second_table.order_id;


--TYPE 2: REMOVE EXTRA SPACES
UPDATE retail_sales_clean
SET
customer_name = TRIM(customer_name),
employee_name = TRIM(employee_name),
department = TRIM(department),
employee_region = TRIM(employee_region),
product_name = TRIM(product_name),
category = TRIM(category);

--TYPE 3: STANDARDIZE TEXT CASE
UPDATE retail_sales_clean
SET
employee_region = INITCAP(LOWER(employee_region)),
product_name = INITCAP(LOWER(product_name)),
department = INITCAP(LOWER(department)),
customer_name = INITCAP(LOWER(customer_name));
```

```sql
--TYPE 4: FIX INCONSISTENT VALUES (SAME CATEGORY WRITTEN DIFFERENTLY

--Cleaning category column
SELECT DISTINCT category
FROM retail_sales_clean
ORDER BY category;

UPDATE retail_sales_clean
SET category='Smartphone'
WHERE category IN ('Smartphone', 'smart-phone', 'Smart phone');

UPDATE retail_sales_clean
SET category='Accessories'
WHERE category IN ('Accessory','accessory', 'accessories');

UPDATE retail_sales_clean
SET category = 'Smartwatch'
WHERE category IN ('smartwatch', 'Smart watch', 'Smart Watch');

--Cleaning Department column
SELECT DISTINCT department
FROM retail_sales_clean
ORDER BY department;

--Cleaning employee region column (this we alredy cleaned in type 2 and 3 cleaning)
SELECT DISTINCT employee_region
FROM retail_sales_clean
ORDER BY employee_region;

--Cleaning product names column
SELECT DISTINCT product_name
FROM retail_sales_clean
ORDER BY product_name;

UPDATE retail_sales_clean
SET product_name = 'Iphone15'
WHERE product_name IN ('Iphone15', 'Iphone 15');

--TYPE 5: HANDLE MISSING VALUES
SELECT * FROM retail_sales_clean
WHERE department = NULL;

--Replace missing values
UPDATE retail_sales_clean
SET department = 'Unknown'
WHERE department = NULL;
```

```sql
--TYPE 6: INCONSISTENT DATE FORMATS
SELECT DISTINCT order_date
FROM retail_sales_clean;

----ADD new date column
ALTER TABLE retail_sales_clean
ADD COLUMN new_order_date DATE;

----Convert differet date formats
UPDATE retail_sales_clean
SET new_order_date =
CASE
    -- Handles dates like 3/5/2024 or 03/05/2024
    WHEN order_date ~ '^[0-9]{1,2}/[0-9]{1,2}/[0-9]{4}$'
    THEN TO_DATE(order_date,'DD/MM/YYYY')

    -- Handles dates like 2024-3-9 or 2024-03-09
    WHEN order_date ~ '^[0-9]{4}-[0-9]{1,2}-[0-9]{1,2}$'
    THEN TO_DATE(order_date,'YYYY-MM-DD')

    -- Handles dates like 7-2-2024 or 07-02-2024
    WHEN order_date ~ '^[0-9]{1,2}-[0-9]{1,2}-[0-9]{4}$'
    THEN TO_DATE(order_date,'DD-MM-YYYY')

    -- Handles dates like 2024/3/9
    WHEN order_date ~ '^[0-9]{4}/[0-9]{1,2}/[0-9]{1,2}$'
    THEN TO_DATE(order_date,'YYYY/MM/DD')
    ELSE NULL
END;

--Drop old date column (optional)
ALTER TABLE retail_sales_clean
DROP COLUMN order_date;


--TYPE 7: FIX WRONG DATATYPES (currently all columns have Text datatype)

ALTER TABLE retail_sales_clean
ALTER COLUMN salary TYPE NUMERIC(10,2)
USING salary::NUMERIC(10,2);

ALTER TABLE retail_sales_clean
ALTER COLUMN order_id TYPE INT
USING order_id::INT;

ALTER TABLE retail_sales_clean
ALTER COLUMN product_id TYPE INT
USING product_id::INT;

ALTER TABLE retail_sales_clean
ALTER COLUMN quantity TYPE INT
USING quantity::INT;

ALTER TABLE retail_sales_clean
ALTER COLUMN product_price TYPE NUMERIC(10,2)
USING product_price::NUMERIC(10,2);
```

```sql
SELECT * FROM retail_sales_clean;

ALTER TABLE retail_sales_clean
ALTER COLUMN sales TYPE NUMERIC(10,2)
USING sales::NUMERIC(10,2);

ALTER TABLE retail_sales_clean
ALTER COLUMN employee_id TYPE INT
USING employee_id::INT;

--TYPE 8: CHECK NEGATIVE VALUES
UPDATE retail_sales_clean
SET sales=product_price*quantity;

UPDATE retail_sales_clean
SET sales=NULL
WHERE sales<0;

--DATA CLEANING COMPLETED
SELECT * FROM retail_sales_clean;
```

5. Database Normalization

Why Normalization?

Normalization is performed to:

- Reduce data redundancy
- Improve data consistency
- Avoid update anomalies
- Improve database efficiency
- Maintain data integrity

Normalization Level:

The database is normalized up to: Third Normal Form (3NF)

This ensures:

- No repeating groups
- No partial dependency
- No transitive dependency

6. Database Schema Design

The cleaned dataset was divided into three tables:

1. employees
2. products
3. sales

7. PostgreSQL Table Creation Queries

Create Employees Table

```
CREATE TABLE employee (
  employee_id INT PRIMARY KEY,
  employee_name TEXT,
  employee_region TEXT,
  salary NUMERIC(10,2),
  department TEXT);
```

Create Products Table

```
CREATE TABLE product (
  product_id INT PRIMARY KEY,
  product_name TEXT,
  product_price NUMERIC(10,2),
  category TEXT);
```

Create Sales Table

```
CREATE TABLE sales (
  order_id INT PRIMARY KEY,
  sales NUMERIC(10,2),
  quantity INT,
  order_date DATE,
  customer_name TEXT,
  employee_id INT,
  product_id INT,

  FOREIGN KEY(employee_id)
  REFERENCES employee(employee_id),

  FOREIGN KEY(product_id)
  REFERENCES product(product_id));
```

8. Data Insertion Queries

Insert Data into Employees Table, Product Table and Sales Table

```
INSERT INTO employee (employee_id, employee_name, employee_region, salary, department)
SELECT DISTINCT employee_id, employee_name, employee_region, salary, department
FROM retail_sales_clean;

INSERT INTO product (product_id, product_name, product_price, category)
SELECT DISTINCT product_id, product_name, product_price, category
FROM retail_sales_clean;

INSERT INTO sales (order_id, sales, quantity, order_date, customer_name, employee_id, product_id)
SELECT DISTINCT order_id, sales, quantity, new_order_date, customer_name, employee_id, product_id
FROM retail_sales_clean;
```

9. ER Diagram Structure

The following ER diagram represents the normalized database structure of the AniRetail project.

Tables:

- employee
- product
- sales

Relationships:

- One employee can perform multiple sales
- One product can appear in multiple sales transactions
- Sales table acts as the central transactional table

employees (1) -------- (M) sales (M) -------- (1) products

10. Business Analysis Questions with SQL Queries

1. Find total sales generated by each employee

```sql
SELECT e.employee_name, SUM(s.sales) AS total_sales
FROM sales s
JOIN employee e
ON e.employee_id = s.employee_id
GROUP BY e.employee_name
ORDER BY total_sales DESC;
```

Output:

	employee_name text	total_sales numeric
1	Priya Mehta	2613000.00
2	Simran Kaur	2348000.00
3	Rahul Sharma	2046000.00
4	Vikas Rao	2020000.00
5	Amit Verma	1675000.00
6	Sneha Kapoor	1377000.00
7	Rohan Gupta	1077000.00
8	Neha Joshi	726000.00

2. Find top-performing employee

```sql
SELECT e.employee_name, Sum(s.sales) AS total_sales
FROM employee e
JOIN sales s
ON e.employee_id=s.employee_id
GROUP BY e.employee_name
ORDER BY total_sales DESC LIMIT 1;
```

Output:

	employee_name text	total_sales numeric
1	Priya Mehta	2613000.00

3. Find employee-wise order count

```sql
SELECT e.employee_name, COUNT(s.order_id) AS total_orders
FROM sales s
JOIN employee e
ON e.employee_id = s.employee_id
GROUP BY e.employee_name
ORDER BY total_orders DESC;
```

Output:

	employee_name text	total_orders bigint
1	Vikas Rao	17
2	Simran Kaur	15
3	Sneha Kapoor	14
4	Rahul Sharma	13
5	Priya Mehta	12
6	Amit Verma	12
7	Neha Joshi	10
8	Rohan Gupta	7

4. Find region-wise sales

```sql
SELECT e.employee_region, SUM(s.sales) AS total_sales
FROM sales s
JOIN employee e
ON e.employee_id = s.employee_id
GROUP BY e.employee_region
ORDER BY total_sales DESC;
```

Output:

	employee_region text	total_sales numeric
1	East	3725000.00
2	West	3695000.00
3	South	3339000.00
4	North	3123000.00

5. Find best-selling products

```sql
SELECT p.product_name, SUM(s.quantity) AS total_quantity
FROM product p
JOIN sales s
ON p.product_id=s.product_id
GROUP BY p.product_name
ORDER BY total_quantity DESC;
```

Output:

	product_name text	total_quantity bigint
1	Boat Headphon...	56
2	Iphone15	51
3	Samsung S24	44
4	Sony Headphon...	37
5	Noise Watch	36
6	Dell Inspiron	33
7	Macbook Air	30
8	Apple Watch	28

6. Find highest revenue generating products

```
SELECT p.product_name, SUM(s.sales) AS total_revenue
FROM product p
JOIN sales s
ON p.product_id=s.product_id
GROUP BY p.product_name
ORDER BY total_revenue DESC LIMIT 1;
```

Output:

	product_name text	total_revenue numeric
1	Iphone15	4080000.00

7. Find category-wise sales

```
SELECT p.category, SUM(s.sales) AS total_sales
FROM product p
JOIN sales s
ON p.product_id=s.product_id
GROUP BY p.category
ORDER BY total_sales DESC;
```

Output:

	category text	total_sales numeric
1	Smartphone	7160000.00
2	Laptop	4995000.00
3	Smartwatch	1300000.00
4	Accessories	427000.00

8. Find average product price by category

```
SELECT category, AVG(product_price) AS average_price
FROM product GROUP BY category;
```

Output:

	category text	average_price numeric
1	Smartphone	75000.000000000000
2	Smartwatch	22500.000000000000
3	Accessories	5000.0000000000000000
4	Laptop	80000.000000000000

9. Find total company revenue

```
SELECT SUM(sales) AS total_revenue FROM sales;
```

Output:

	total_revenue numeric
1	13882000.00

10. Find monthly sales trend

```
SELECT DATE_TRUNC('month',order_date) AS month, SUM(sales) AS total_sales
FROM sales
GROUP BY month
ORDER BY month;
```

Output:

	month timestamp with time zone	total_sales numeric
1	2024-01-01 00:00:00+05:30	5190000.00
2	2024-02-01 00:00:00+05:30	5176000.00
3	2024-03-01 00:00:00+05:30	3516000.00

11. Find highest sales day

```sql
SELECT order_date, SUM(sales) AS total_sales
FROM sales
GROUP BY order_date
ORDER BY total_sales DESC LIMIT 1;
```

Output:

	order_date date	total_sales numeric
1	2024-01-10	950000.00

12. Find average order value

```sql
SELECT AVG(sales) AS average_order_value
FROM sales;
```

Output:

	average_order_value numeric
1	138820.000000000000

13. Find top 5 highest sales orders

```sql
SELECT order_id, sales AS top_five_sales
FROM sales
ORDER BY top_five_sales DESC LIMIT 5;
```

Output:

	order_id [PK] integer	top_five_sales numeric (10,2)
1	1033	475000.00
2	1087	475000.00
3	1089	475000.00
4	1074	400000.00
5	1027	400000.00

14. Find total quantity sold by category

```
SELECT p.category, SUM(s.quantity) AS total_quantity
FROM product p
JOIN sales s
ON p.product_id = s.product_id
GROUP BY category
ORDER BY total_quantity DESC;
```

Output:

	category text	total_quantity bigint
1	Smartphone	95
2	Accessories	93
3	Smartwatch	64
4	Laptop	63

15. Find employees who sold the most products

```
SELECT e.employee_name, SUM(s.quantity) AS no_of_products_sold
FROM employee e
JOIN sales s
ON e.employee_id = s.employee_id
GROUP BY e.employee_id
ORDER BY no_of_products_sold DESC LIMIT 1;
```

Output:

	employee_name text	no_of_products_sold bigint
1	Vikas Rao	54

16. Find contribution percentage of each category

```
SELECT p.category,
    ROUND(SUM(s.sales) * 100.0 / (SELECT SUM(sales) FROM sales),2)
    AS contribution_percentage
FROM sales s
JOIN products p
ON s.product_id = p.product_id
GROUP BY p.category
ORDER BY contribution_percentage DESC;
```

Output:

	category text	contribution_percentage numeric
1	Smartphone	51.58
2	Laptop	35.98
3	Smartwatch	9.36
4	Accessories	3.08

17. Find running total of sales

```
SELECT order_date, sales, SUM(sales) OVER(ORDER BY order_date) AS running_total
FROM sales;
```

Output:

	order_date date	sales numeric (10,2)	running_total numeric
1	2024-01-03	7000.00	7000.00
2	2024-01-06	28000.00	175000.00
3	2024-01-06	140000.00	175000.00
4	2024-01-08	400000.00	655000.00
5	2024-01-08	80000.00	655000.00
6	2024-01-09	475000.00	1361000.00
7	2024-01-09	21000.00	1361000.00
8	2024-01-09	210000.00	1361000.00
9	2024-01-10	320000.00	2311000.00
10	2024-01-10	25000.00	2311000.00
11	2024-01-10	320000.00	2311000.00
12	2024-01-10	285000.00	2311000.00
13	2024-01-11	9000.00	2320000.00
14	2024-01-13	140000.00	2460000.00

18. Rank employees based on sales

```
SELECT e.employee_name, SUM(s.sales) AS total_sales,
  RANK() OVER(ORDER BY SUM(s.sales) DESC)
FROM sales s
JOIN employee e
ON e.employee_id = s.employee_id
GROUP BY e.employee_name;
```

Output:

	employee_name text	total_sales numeric	rank bigint
1	Priya Mehta	2613000.00	1
2	Simran Kaur	2348000.00	2
3	Rahul Sharma	2046000.00	3
4	Vikas Rao	2020000.00	4
5	Amit Verma	1675000.00	5
6	Sneha Kapoor	1377000.00	6
7	Rohan Gupta	1077000.00	7
8	Neha Joshi	726000.00	8

19. Find most profitable category

```
SELECT p.category, SUM(s.sales) AS total_sales
FROM sales s
JOIN product p
ON p.product_id = s.product_id
GROUP BY p.category
ORDER BY total_sales DESC LIMIT 1;
```

Output:

	category text	total_sales numeric
1	Smartphone	7160000.00

20. Find products never sold

```
SELECT p.product_name
FROM product p
LEFT JOIN sales s
ON p.product_id = s.product_id
WHERE s.quantity <1;
```

Output:

	product_name text

11. Key Insights:

Example Insights from AniRetail Project:

- Certain product categories contribute significantly more revenue than others.
- Some employees consistently outperform others in sales generation.
- Monthly sales trends reveal peak and low-performing periods.
- Regional analysis helps identify high-performing markets.
- Product-wise analysis identifies best-selling products.
- SQL joins help combine transactional and master data for deeper insights.

After analysis, the company can identify:

- Top-performing employees
- Best-selling products
- High-revenue product categories
- Regional sales performance
- Monthly sales growth trends
- Customer purchasing behavior

These insights help management improve decision-making and business strategy.

12. Conclusion

The AniRetail project successfully demonstrates a complete end-to-end PostgreSQL data analytics workflow.

The project involved:

- Raw data cleaning

- Data standardization
- Database normalization
- PostgreSQL table creation
- Data insertion
- SQL-based business analysis

The final system provides structured, reliable, and analysis-ready retail data for business intelligence purpose.
